Report: Enhancements to Mail System Model

Project: M.EIC037 Mini Project 1
Group 3: José Felisberto, Bruno Machado, Pedro Romão
Date: 2025-04-07

Overview

This report details enhancements made to the Alloy model. These include incorporating message "read" status tracking and enforcing strict rules for message status transitions (`Fresh` , `External` , `Active` , `Purged`).

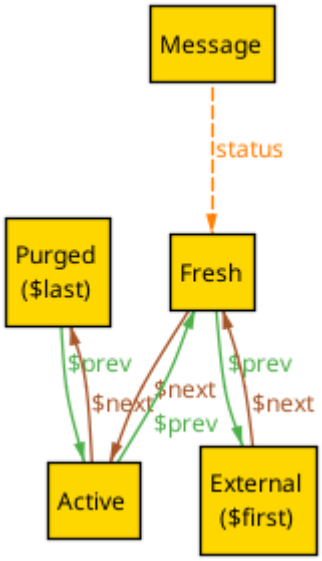
Core Changes

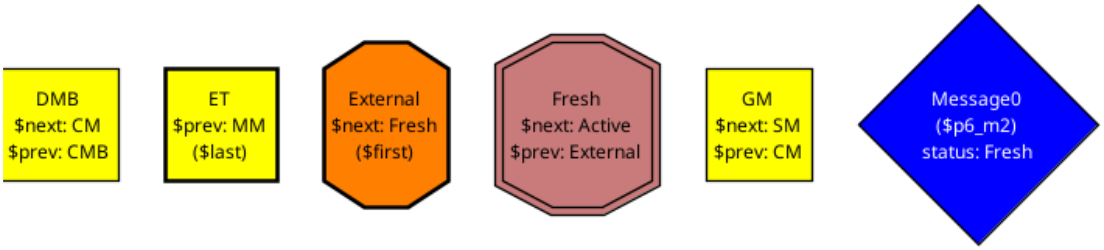
1. **Strict Status Transition Rules (`ValidStatusTransitions`)**
   - A `fact` named `ValidStatusTransitions` has been added under "Improvements" to enforce specific, valid transitions between message statuses globally across all operations:

     ```
     fact ValidStatusTransitions {
       always (
         all m: Message |
           // Fresh can only become Active
           (m.status = Fresh and m.status' != Fresh) => m.status' =
           and
           // External can only become Active
           (m.status = External and m.status' != External) => m.stat
           and
           // Active can only become Purged
           (m.status = Active and m.status' != Active) => m.status'
           and
           // Purged stays Purged
           (m.status = Purged) => m.status' = Purged
       )
     }
     ```

   - This ensures messages follow a defined lifecycle:
     - `Fresh` → `Active`
     - `External` → `Active`

    - `Active` → `Purged`
    - `Purged` remains `Purged`
  - This prevents weird transitions (e.g., `External` to `Fresh`, etc.) that might otherwise be possible.





2. **Message "Read" Status Tracking:**
  - The `Message` signature now includes a variable `read` of type `lone Boolean`:

```
sig Message extends Object {
  var status: lone Status,
  var read: lone Boolean  // Added
}
```

- This allows each message to optionally track whether it has been read (True) or not (False).

3. **Initial State:**
   - The `Init` predicate ensures all messages start in the `Fresh` state (as per original model structure, implied) and explicitly sets the initial `read` state:

```
pred Init {
  all m: Message | m.status = Fresh
  all m: Message | m.read = False // Added
  // ... other initial conditions ...
}
```

4. **New Operator: `readMessage` (`RM`)**
   - A new operator `readMessage[m: Message]` has been defined (tracked by `RM` in the `Operator` enum).
   - **Preconditions:**
     - The message `m` must be `Active` .
     - The message `m` must currently be unread (`m.read = False`).
     - The message `m` must exist in *some* mailbox.
   - **Postconditions:**
     - The message's read status is set to `True` (`m.read' = True`).
   - **Frame Conditions:**
     - The status (`Active` , `Purged` , etc.) of all messages remains unchanged (`noStatusChange[Message]`). This is consistent with `ValidStatusTransitions` as no status change occurs.
     - Messages do not change mailboxes (`noMessageChange[Mailbox]`).

    - The set of user-defined mailboxes remains unchanged (`noUserboxChange`).
    - The read status of *other* messages remains unchanged (`all msg: Message - m | msg.read' = msg.read`).

5. **System Integration & Frame Conditions:**
    - The `RM` operator was added to the `Operator` enum.
    - The `Trans` predicate now includes `(some m: Message | readMessage[m])` allowing this operation in system traces.
    - A helper predicate `noReadChange` was added to express that the read status of a set of messages remains unchanged.
    - Existing operators `emptyTrash` and `deleteMailbox` were updated to explicitly preserve the read status of messages being purged (consistent with `Purged` state persistence in `ValidStatusTransitions`).
    - The `noOp` predicate was updated to preserve the read status.

Verification & Properties

1. **Sanity Check (`readMessageWorks`):**
    - A predicate `readMessageWorks` was added to check if it's possible for a message to eventually be marked as read.
    - `run readMessageWorks for 5 but 11 Object` confirms this scenario is reachable within the specified scope.

2. **Persistence Property (`readPersistence`):**
    - An assertion `readPersistence` was added to check if a message, once marked as read, remains read (until potentially purged).

```
assert readPersistence {
  all m: Message |
    once (m.read = True) =>
    always (m.read = True)
}
```

- `check readPersistence for 5 but 11 Object` indicates that within the checked scope, no counterexample was found where a message becomes unread after being read. This holds true as the `readMessage` operation only sets it to `True`, and other operations like `emptyTrash` / `deleteMailbox` preserve the value during the transition to `Purged`.

Summary

The model now successfully incorporates the concept of messages being marked as "read" and enforces a strict lifecycle for message statuses via the `ValidStatusTransitions` fact.

33 commands were executed. The results are:

#1: **Instance found.** run$1 is consistent.
#2: **Instance found.** p1 is consistent.
#3: **Instance found.** p2 is consistent.
#4: **Instance found.** p3 is consistent.
#5: **Instance found.** p4 is consistent.
#6: **Instance found.** p5 is consistent.
#7: **Instance found.** p6 is consistent.
#8: **Instance found.** p7 is consistent.
#9: **Instance found.** p8 is consistent.
#10: **Instance found.** p9 is consistent.
#11: **Instance found.** p10 is consistent.
#12: **Instance found.** readMessageWorks is consistent.
#13: No counterexample found. v1 may be valid.
#14: No counterexample found. v2 may be valid.
#15: No counterexample found. v3 may be valid.
#16: No counterexample found. v4 may be valid.
#17: No counterexample found. v5 may be valid.
#18: No counterexample found. v6 may be valid.
#19: No counterexample found. v7 may be valid.
#20: No counterexample found. v8 may be valid.
#21: No counterexample found. v9 may be valid.
#22: No counterexample found. v10 may be valid.
#23: No counterexample found. v11 may be valid.
#24: No counterexample found. v12 may be valid.
#25: No counterexample found. v13 may be valid.
#26: No counterexample found. v14 may be valid.
#27: No counterexample found. v15 may be valid.
#28: No counterexample found. v16 may be valid.
#29: **Counterexample found.** i1 is invalid.
#30: **Counterexample found.** i2 is invalid.
#31: **Counterexample found.** i3 is invalid.
#32: **Counterexample found.** i4 is invalid.
#33: No counterexample found. readPersistence may be valid.